Mathematics behind Linear Regression and Logistic Regression

Khiem Nguyen Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

Table of Content

1 Linear Regression with multiple features: Theory **2**
1.1 Problem description . . . 2
1.2 Linear regression model . . . 2
1.3 Loss function . . . 2
1.4 Gradient of loss function . . . 3
1.5 Numerical Example . . . 3
1.5.1 Hand-written calculation . . . 4
1.5.2 Python implementation using symbolic computation . . . 6

2 Logistic regression for binary classification **11**
2.1 Problem description . . . 11
2.2 Logistic regression model for binary classification . . . 13
2.3 Loss function for training the logistic regression model . . . 15
2.4 Regularization for logistic regression . . . 16
2.5 Gradient of loss function . . . 20
2.6 Update rule for gradient descent . . . 20

3 Logistic regression for multi-class classification **21**
3.1 Problem description . . . 21
3.2 Logistic regression model for multi-class classification . . . 21
3.3 Loss function for training the logistic regression model for multi-class classification . . . 22
3.4 Loss function for regularized logistic regression for multi-class classification . . . 23
3.5 Gradient of loss function for multi-class classification . . . 24
3.6 Update rule for gradient descent for multi-class classification . . . 24

Equations from main file `assignment_02.html`

General Dataset

We are given a dataset $\mathcal{D}$ with m samples and each sample has n input features $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)}) \in \mathbb{R}^n$ and one output label $y \in \mathbb{R}^n$.

$$\mathcal{D} = \left\{ (\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)}) \right\} \tag{1}$$

Gradient Descent Update Formla

The gradient descent method is given by the following iteration formula. The variable Θ corresponds to the model parameters, which are the learnable parameters of the model. The learning rate α is a positive scalar hyperparameter that controls the step size of each iteration.

$$\Theta^{[t+1]} := \Theta^{[t]} - \alpha \nabla \mathcal{L}(\Theta^{[t]}), \quad \forall t = 0, 1, 2, \dots \tag{2}$$

1 Linear Regression with multiple features: Theory

In this section, we study:

- Description of a regresison problem
- Linear regression model
- Loss function for training the linear regression model
- Gradient of loss function

1.1 Problem description

We are given a dataset $\mathcal{D}$ with m samples and each sample has n input features $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)}) \in \mathbb{R}^n$ and one output label $y \in \mathbb{R}^n$ – see Equation 1 again

$$\mathcal{D} = \left\{ (\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)}) \right\} \tag{3}$$

This dataset can be written in two matrices

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times n}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \in \mathbb{R}^m \tag{4}$$

Our task is to build a model, which is actually just a function or a mapping, to predict a new output $\hat{y}$ given a new sample (data point) $\mathbf{x}$. That is we need to construct the function $f = f(\mathbf{x})$ so that it fits the data very well. If the model can be visualized, we expect the graph $y = f(\mathbf{x})$ should fit the data points in the high-dimensional space $\mathbf{R}^{n+1}$ very well. Now, we go through three steps mentioned in section "Build and Train ML models"

1.2 Linear regression model

The linear regression model is defined by a simple linear relationship between input $\mathbf{x}$ and output y:

$$\hat{y} = f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b.$$

The parameters $\mathbf{w} = (w_1, \dots, w_n)$ are called **weights** and the parameter b is called **bias**. This linear regression model is fully determined by the **learnable parameters** $\mathbf{w} = (w_1, \dots, w_n)$ and b. **Our objective** is to seek these *learnable parameters* $w_1, \dots, w_n$ and b.

1.3 Loss function

To measure how well the model is, we use the **Mean Square Error** (MSE) loss function:

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b) &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2 = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2 \\ &= \frac{1}{m} \sum_{i=1}^{m} \left(\mathbf{w} \cdot \mathbf{x}^{(i)} + b - y^{(i)}\right)^2 \end{aligned} \tag{5}$$

Note that we don't use $2m$ in the denominator as in the Assignment 01. This is just to prove that the minimium point is exactly the same, only the minimum value is scaled by a factor $1/2$. For this reason, the gradient $\nabla \mathcal{L}$ will be also scaled (see below).

It is immediately clear that the loss function $\mathcal{L}$ defined as above is just a function of $\mathbf{w}$ and b because the values $\mathbf{x}^{(i)}$ and $y^{(i)}$ for all $i = 1, \dots, m$ are given in the dataset $\mathcal{D}$ (e.g., in the `.csv` file).

We recall here "training the model" means "looking for $\mathbf{w}$ and b" and the way to do this is to implement the gradient descent method given by the iteration formula Equation 2. In the current case, the variable Θ corresponds to the model parameters $\mathbf{w}$ and b. To implement the update rule given by Equation 2, we need to compute the gradient $\nabla\mathcal{L}$ which refers to derivatives of $\mathcal{L}$ with respect to $w_1, w_2, \dots, w_n$ and to b:

$$\nabla\mathcal{L} = \left(\frac{\partial\mathcal{L}}{\partial w_1}, \cdots, \frac{\partial\mathcal{L}}{\partial w_n}, \frac{\partial\mathcal{L}}{\partial b}\right)$$

Therefore, we need to compute the gradient of the loss function $\nabla\mathcal{L}$ to perform the gradient descent iteration algorithm.

1.4 Gradient of loss function

The gradient of $\mathcal{L}$ given by Equation 5 is computed as follows[1] – If you are interested in the derivation, please refer `derivation.html`.

$$\frac{\partial\mathcal{L}}{\partial w_1} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})x_1^{(i)}$$

so on

$$\frac{\partial\mathcal{L}}{\partial w_n} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})x_n^{(i)}$$

$$\frac{\partial\mathcal{L}}{\partial b} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot 1$$

Note that since we have m instead of $2m$ in the denominator of Equation 5, the gradient is also scaled, and thus we see number 2 in the nominator of $\frac{2}{m}$ as compared to $\frac{1}{m}$ in the Assignment 01. By plugging these equations in Equation 2, we obtain the following update rule

$$w_1^{[t+1]} := w_1^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[\left(\mathbf{w}^{[t]} \cdot \mathbf{x}^{(i)} + b^{[t]}\right) - y^{(i)}\right]x_1^{(i)}$$

so on

$$w_n^{[t+1]} := w_n^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[\left(\mathbf{w}^{[t]} \cdot \mathbf{x}^{(i)} + b^{[t]}\right) - y^{(i)}\right]x_n^{(i)}$$

$$b^{[t+1]} := b^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[\left(\mathbf{w}^{[t]} \cdot \mathbf{x}^{(i)} + b^{[t]}\right) - y^{(i)}\right] \cdot 1$$

1.5 Numerical Example

The Black Box Trap In modern engineering and physics, it is easy to feed parameters into a simulation or a Python script and get a result. However, this normally leads to the feeling that "we understand it" while in reality "we don't quite really understand it."

> Tip
>
> In **MIT** or **CalTech** you will frequently hear professors and Teaching Assitants reiterate some version of "If you cannot solve it for $n = 2$, you don't understand the general case."

[1] You can try to carry out the derivation. It is surprisingly easy to be honest. Clever students like you will definitely do this

For this reason, let us go through all the theoretical steps above using a simple 2-dimensional example with 4 samples. Note that with only 3 points, we can always build a plane that go through all 3 points. Thus, we chose 4 points.

Dataset

For reproducibility purpose, we design the dataset by using the exact/reference linear regression model:

$$f(x_1, x_2) = \frac{1}{2}x_1 - \frac{1}{2}x_2 + \frac{3}{2} \tag{6}$$

The dataset is given by the following table

x_1	x_2	y
0	0	3/2
1	0	2
0	1	1
1	1	v

We keep the label value of the last data point general, denoted by v. We shall see that by adjusting v, the loss function will also change and the minimum point is not always equal to 0.

1.5.1 Hand-written calculation

Linear Regression Model

In this example, we have 2 features. The linear regression model for 2 input features reads

$$\hat{y} = f_{\mathbf{w},b}(x_1, x_2) = w_1 x_1 + w_2 x_2 + b.$$

Loss Function

We reiterate the formula for the loss function

$$\begin{aligned}\mathcal{L}(w_1, w_2, b) &= \frac{1}{4}\sum_{i=1}^{4}\left(\hat{y}^{(i)} - y^{(i)}\right)^2 = \frac{1}{4}\sum_{i=1}^{4}\left[e^{(i)}\right]^2 \\ &= \frac{1}{4}\sum_{i=1}^{4}\left(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b - y^{(i)}\right)^2\end{aligned} \tag{7}$$

where the error term $e^{(i)}$ is defined by

$$\begin{aligned}e^{(i)} &= \hat{y}^{(i)} - y^{(i)} = f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)} \\ &= w_1 x_1^{(i)} + w_2 x_2^{(i)} + b - y^{(i)}.\end{aligned}$$

For the dataset with $m = 4$ samples, we calculate the individual squared errors by direct substitution:

$$\text{For } (0, 0, 3/2) : e^{(1)} = (b - 3/2)$$

$$\text{For } (1, 0, 2) : e^{(2)} = (w_1 + b - 2)$$

$$\text{For } (0, 1, 1) : e^{(3)} = (w_2 + b - 1)$$

$$\text{For } (1, 1, v) : e^{(4)} = (w_1 + w_2 + b - v)$$

Substituting these expressions for $e^{(i)}$ into Equation 7, we arrive at the total loss function

$$\mathcal{L}(w_1, w_2, b) = \frac{1}{4}\left[\left(b - \frac{3}{2}\right)^2 + (w_1 + b - 2)^2 + (w_2 + b - 1)^2 + (w_1 + w_2 + b - v)^2\right]$$

Note It is clear that no matter how complicated the dataset is, the loss function for linear regression models is a quadratic function with respect to the model parameters. In this case, $\mathcal{L}$ is a quadratic function of (w_1, w_2, b). For this reason, there exists one and only one minimal point $\mathcal{L}$, which is computed by solving the equation $\nabla\mathcal{L} = 0$.

Gradient of the Loss Function

The mimization of $\mathcal{L}$ can be obtained by solving the equation $\nabla\mathcal{L} = 0$, namely the equations retrieved by setting the partial derivative of $\mathcal{L}$ with respect to all the variables to 0. In this case, it simply reads

$$\nabla\mathcal{L} = 0 \quad \Leftrightarrow \quad \begin{cases} \dfrac{\partial\mathcal{L}}{\partial w_1} = 0, \\ \dfrac{\partial\mathcal{L}}{\partial w_1} = 0, \\ \dfrac{\partial\mathcal{L}}{\partial b} = 0 \end{cases} \tag{8}$$

We can easily derive

$$\begin{aligned} \frac{\partial\mathcal{L}}{\partial w_1} &= \frac{1}{2}\left[(w_1 + b - 2) + (w_1 + w_2 + b - v)\right], \\ \frac{\partial\mathcal{L}}{\partial w_2} &= \frac{1}{2}\left[(w_2 + b - 1) + (w_1 + w_2 + b - v)\right], \\ \frac{\partial\mathcal{L}}{\partial b} &= \frac{1}{2}\left[(b - 1.5) + (w_1 + b - 2) + (w_2 + b - 1) + (w_1 + w_2 + b - v)\right] \end{aligned}$$

Then, we obtain a system of 3 equations for 3 **learnable parameters** w_1, w_2, b. The three equations given by the optimality condition Equation 8 have one single solution:

$$w_1 = \frac{2v - 1}{4}, \quad w_2 = \frac{2v - 5}{4}, \quad b = \frac{15 - 2v}{8} \tag{9}$$

Discussion on the loss function

By substituting Equation 9 into the loss function Equation 7, we obtain the minimum of the loss function – note that this minimum value still depends on v because we allow the forth data points to move freely in the y-axis with value v:

Substituting the optimal parameters back into the error terms (without being squared here):

$$\begin{aligned} e^{(1)} &= \left(\frac{2v-1}{4}\right)\cdot 0 + \left(\frac{2v-5}{4}\right)\cdot 0 + \left(\frac{15-2v}{8}\right) - \frac{3}{2} = \frac{3-2v}{8} \\ e^{(2)} &= \left(\frac{2v-1}{4}\right)\cdot 1 + \left(\frac{2v-5}{4}\right)\cdot 0 + \left(\frac{15-2v}{8}\right) - 2 = \frac{2v-3}{8} \\ e^{(3)} &= \left(\frac{2v-1}{4}\right)\cdot 0 + \left(\frac{2v-5}{4}\right)\cdot 1 + \left(\frac{15-2v}{8}\right) - 1 = \frac{2v-3}{8} \\ e^{(4)} &= \left(\frac{2v-1}{4}\right)\cdot 1 + \left(\frac{2v-5}{4}\right)\cdot 1 + \left(\frac{15-2v}{8}\right) - v = \frac{3-2v}{8} \end{aligned}$$

The loss function evaluates to

$$\mathcal{L}_{\min} = \frac{1}{4}\sum_{i=1}^{4}\left[e^{(i)}\right]^2 = \frac{(2v-3)^2}{64}$$

> **Note**
>
> **The minimum of a loss function does not need to be zero**

As we can see the minimal loss function $\mathcal{L}_{\min}$ can be non-zero depending on v. The next legitimate question is that what it implies when $\mathcal{L}_{\min} = 0$. Let us think of this scenario.

The loss function measures how well our prediction model makes prediction on the seen/given data. The zero loss function implies that the model has made perfect prediction on the seen/given dataset because all the error terms $e^{(i)}$ must be equal to zero for this to occur. As a consequence, all the data points sit perfectly on the graph of the function

$$(G): \quad y = f_{\mathbf{w},b}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b.$$

For a dataset of 3 data points, we can always construct a plane going through all the data points regardless of the actual values of the data points. This is no longer the case for 4 data points. When $v = 3/2$, the minimum of loss function is equal to zero, $\mathcal{L}_{\min} = 0$. In this case, the 4th data point is given by $(1, 1, 3/2)$, which sits perfectly on the graph – see the reference model Equation 6

$$y = \hat{f}(x_1, x_2) = \frac{1}{2}x_1 - \frac{1}{2}x_2 + \frac{3}{2}$$

because $y^{(4)} = \hat{f}(x_1^{(4)}, x_2^{(4)})$:

$$\frac{3}{2} = \frac{1}{2} \cdot 1 - \frac{1}{2} \cdot 1 + \frac{3}{2}$$

1.5.2 Python implementation using symbolic computation

Instead of computing the loss function by hand, I take this opportunity to introduce you to a useful library in Python: SymPy. To this end, please just follow the code snippet. The code is quite easy to understand.

First of all, you need to activate the conda environment or the virtual environment and then install `sympy` by using one of the following towo command lines:

```
conda install sympy
```

```
pip install sympy
```

Compute loss function

As the starting point, we setup the dataset, define the linear regression model and compute the loss function with the help of symbolic calculation by `sympy` library.

```
import sympy as sp
w1, w2, b, v = sp.symbols('w1 w2 b v')  # Define symbols
data = [(0, 0, 3/2), (1, 0, 2), (0, 1, 1), (1, 1, v)]   # dataset with 4 samples

def model(x1, x2):  # Define the linear regression model
    return w1 * x1 + w2 * x2 + b

squared_errors = [(model(x1, x2) - ylabel)**2 for x1, x2, ylabel in data]
n_samples = len(data)
loss_function = 1 / (n_samples) * sum(squared_errors)

# Display results
def prettify(text: str, n_dashes=30, char="="):
```

```
    long_dash = n_dashes*char
    print("\n" + long_dash + "\n" + text + "\n" + long_dash)

prettify("Symbolic Loss Function")
sp.pprint(loss_function)
```

```
==============================
Symbolic Loss Function
==============================
              2                     2                     2
0.25 (b - 1.5)  + 0.25 (b + w  - 2)  + 0.25 (b + w  - 1)  + 0.25 (b - v + w  +

      2
   w )
```

Compute gradient of loss function and optimal parameters of the linear regression model

Next, we solve the equation $\nabla\mathcal{L} = 0$ for the "learnable parameters", namely the model parameters (w_1, w_2, b). The results are printed for comparison with hand calculation above.

> **Tip**
>
> In fact, I used the results obtained by Python to write down the hand-written computation.

```
grads = [sp.simplify(sp.diff(loss_function, var)) for var in (w1, w2, b)]
gtext = ("dL/dw1", "dL/dw2", "dL/db")
prettify(f"Gradient of the Loss Function")
for text, expr in zip(gtext, grads):
    print(f"{text} = {expr}")

# Solve the system gradient-of-loss = 0
solution = sp.solve(grads, (w1, w2, b))
prettify(f"Optimized model parameters")
for param, expr in solution.items():
    print(f"{param} = {expr}")
```

```
==============================
Gradient of the Loss Function
==============================
dL/dw1 = 1.0*b - 0.5*v + 1.0*w1 + 0.5*w2 - 1.0
dL/dw2 = 1.0*b - 0.5*v + 0.5*w1 + 1.0*w2 - 0.5
dL/db = 2.0*b - 0.5*v + 1.0*w1 + 1.0*w2 - 2.25

==============================
Optimized model parameters
==============================
b = 1.875 - 0.25*v
w1 = 0.5*v - 0.25
w2 = 0.5*v - 1.25
```

Compute minimum of loss function

By direct substitution of the solution obtained above into the loss function `loss`, we can compute the minimum of the loss function

```
# Substitute the solution for w1, w2, b into loss function
min_loss = loss_function.subs(solution)
min_loss_simplified = sp.simplify(min_loss)

prettify(f"Min of loss function")
print(f"L_min = {min_loss_simplified}")

# Solve L_min = 0 for v
vhat = sp.solve(min_loss_simplified, v)[0]
print(f"v (for L_min = 0): {vhat:.3f}")
```

```
==============================
Min of loss function
==============================
L_min = 1.0*(0.25*v - 0.375)**2
v (for L_min = 0): 1.500
```

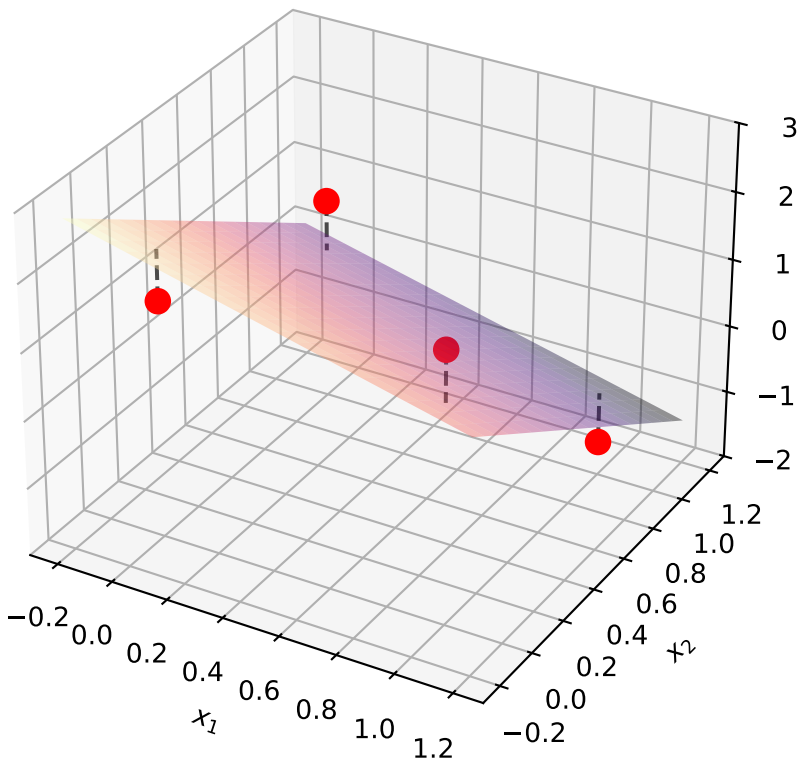
Visualization

Let us visualize the results.

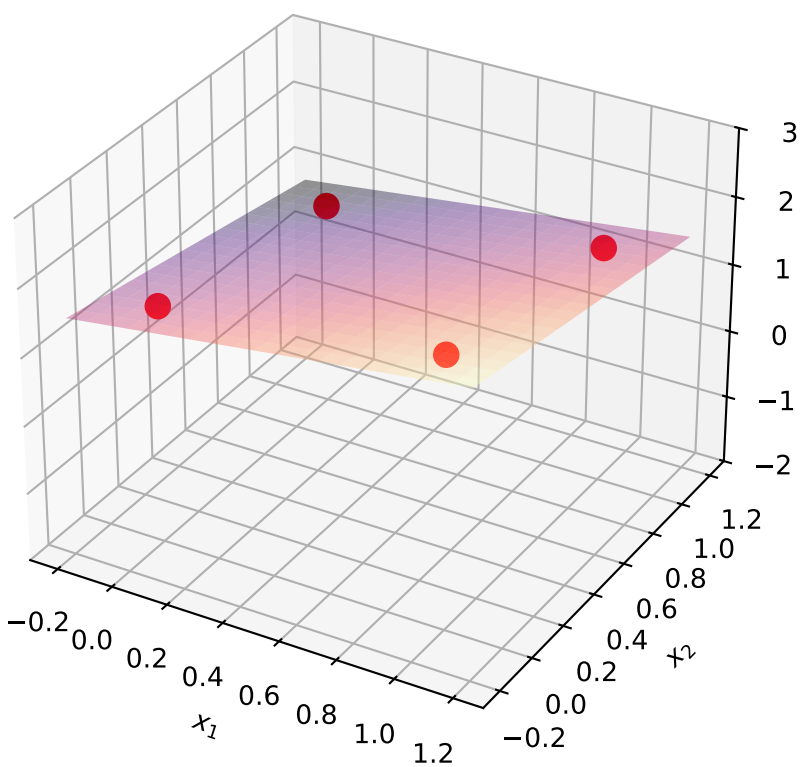
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def plot_regression_3d(v_val, ax, title):
    # Data points
    x1_data = np.array([0, 1, 0, 1])
    x2_data = np.array([0, 0, 1, 1])
    y_data  = np.array([1.5, 2, 1, v_val])

    # Optimal parameters (derived from grad L = 0)
    w1 = (2 * v_val - 1) / 4
    w2 = (2 * v_val - 5) / 4
    b  = (15 - 2 * v_val) / 8

    # 3. Create grid for the plane
    x1_range = np.linspace(-0.2, 1.2, 20)
    x2_range = np.linspace(-0.2, 1.2, 20)
    X1, X2 = np.meshgrid(x1_range, x2_range)
    Y_plane = w1 * X1 + w2 * X2 + b

    surf = ax.plot_surface(X1, X2, Y_plane, alpha=0.4, cmap='magma', edgecolor='none')

    ax.scatter(x1_data, x2_data, y_data, color='red', s=80, depthshade=False, label='Data')

    # Plot residuals (vertical error lines)
    for i in range(len(x1_data)):
        pred = w1 * x1_data[i] + w2 * x2_data[i] + b
        ax.plot([x1_data[i], x1_data[i]], [x2_data[i], x2_data[i]], [y_data[i], pred],
                color='black', linestyle='--', alpha=0.7)

    # Formatting
    ax.set_title(title, fontsize=14)
```

```
    ax.set_xlabel('$x_1$')
    ax.set_ylabel('$x_2$')
    ax.set_zlabel('$y$')
    ax.set_zlim(-2, 3) # Keep scale consistent for comparison

# we want to see both figures in one screen, so we choose a taller
# figure size to fit two 3D plots vertically.
fig = plt.figure(figsize=(4.5, 10))
# Figure 1: v = -1.5 (High Loss)
ax1 = fig.add_subplot(211, projection='3d')
plot_regression_3d(-1.5, ax1, "Linear Regression: $v = -1.5$ (Non-coplanar)")
# Figure 2: v = 1.5 (Zero Loss)
ax2 = fig.add_subplot(212, projection='3d')
plot_regression_3d(1.5, ax2, "Linear Regression: $v = 1.5$ (Perfect Fit)")
plt.tight_layout()
plt.show()
```

Linear Regression: $v = -1.5$ (Non-coplanar)



Linear Regression: $v = 1.5$ (Perfect Fit)



2 Logistic regression for binary classification

In this section, we study:

- Description of a classification problem
- Logistic regression model
- Loss function for training the logistic regression model
- Gradient of loss function

2.1 Problem description

The problem description for a classification problem is quite similar to the description of a regression problem in terms of the given dataset and the ultimate goal prediction, except that the label y is not continous value. Instead, the label y is a discrete value that represents the class membership of the data point. For example, in a binary classification problem, the label y can take values 0 or 1, where 0 represents one class and 1 represents the other class. The dataset $\mathcal{D}$ is given by Equation 1 again, but with the understanding that $y^{(i)}$ is a discrete value representing the class label of the i-th data point $\mathbf{x}^{(i)}$. The goal of a classification problem is to build a model that can predict the class label of a new data point $\mathbf{x}$, which means we want to construct a function $f = f(\mathbf{x})$ that maps input features to class labels.

Simple example and reasoning for logistic regression

Asume we have a dataset of students' study hours per week and their pass/fail status in a semester. The dataset is given by the following table

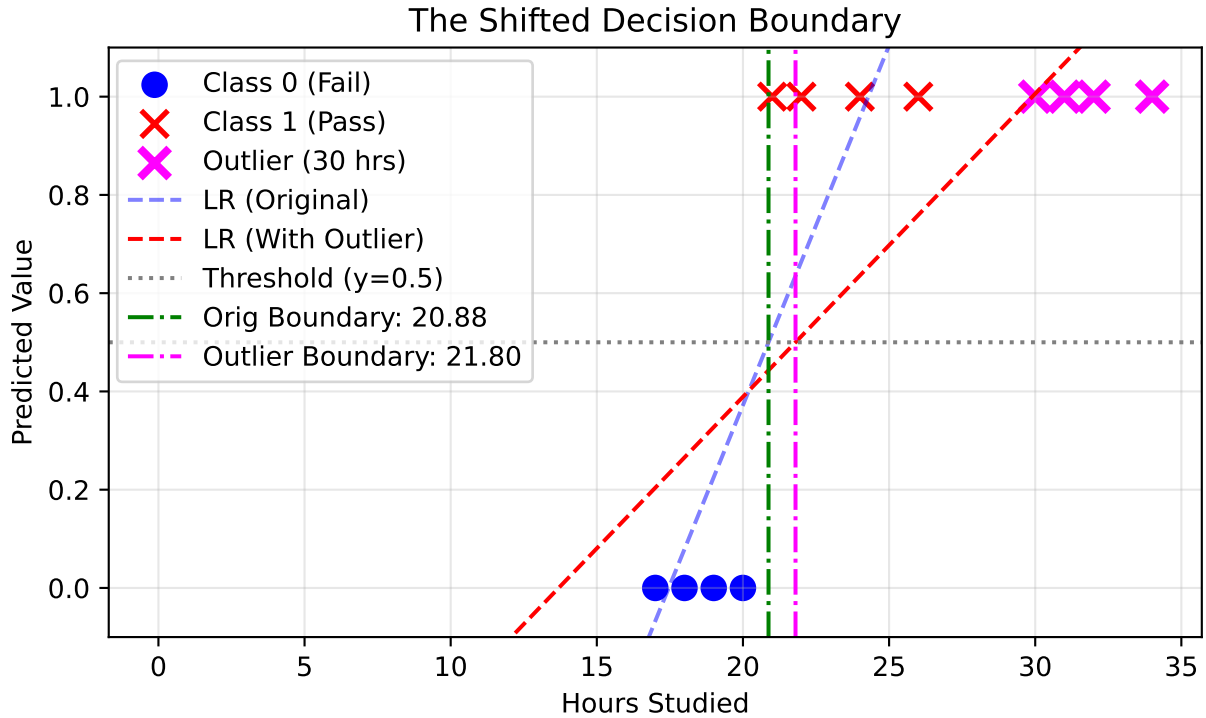
Hours Studied	Pass/Fail (0/1)
17	0
18	0
19	0
20	0
21	1
22	1
23	1
24	1

By looking at the dataset, we can see that there is a clear threshold around 20-21 hours of study. Students who studied less than 21 hours failed (label 0), while those who studied 21 hours or more passed (label 1). If we were to apply linear regression to this dataset, we get a linear line and the decision boundary (where the predicted value is 0.5) would be around 20.5 hours. However, this does not capture the sharp transition from fail to pass that occurs at around 21 hours.

Now, if we introduce outliers: the students who studied 30, 31, 32, and 34 hours and they passed the course with the label 1, the linear regression line would shift significantly, and the decision boundary would move to around 22 hours. This is clearly not desirable as it does not reflect the true nature of the data. Although we call them outliers, they are actually just values that can occur in real life. In real life, we can have more extreme outliers, and thus we need a model that can handle such scenarios without being overly influenced by them. This is where logistic regression comes into play, as it can model the probability of passing as a function of hours studied, and it is less sensitive to outliers compared to linear regression.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
```

```python
X = np.array([17, 18, 19, 20, 21, 22, 24, 26]).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# Not so much outliers, but they are enough to shift the
# decision boundary of linear regression.
outliers = np.array([30, 31, 32, 34]).reshape(-1, 1)
X_with_outlier = np.vstack([X, outliers])
y_with_outlier = np.concatenate([y, [1, 1, 1, 1]])

# Linear Regression using scikit-learn
model_orig = LinearRegression().fit(X, y)
model_outlier = LinearRegression().fit(X_with_outlier, y_with_outlier)

# Calculate X where Y = Threshold (0.5)
# x = (threshold - intercept) / slope
threshold = 0.5
x_bound_orig = (threshold - model_orig.intercept_) / model_orig.coef_[0]
x_bound_outlier = (threshold - model_outlier.intercept_) / model_outlier.coef_[0]

# Plotting: I want to see everything in 1/2 of the monitor
# screen, so I choose a smaller figure size.
plt.figure(figsize=(6.5, 4.0))

# Data points using different markers and colors for classes and outliers
plt.scatter(X[y == 0], y[y == 0], \
    color='blue', label='Class 0 (Fail)', marker='o', s=80)
plt.scatter(X[y == 1], y[y == 1], \
    color='red', label='Class 1 (Pass)', marker='x', s=100, linewidths=2)
plt.scatter(outliers, [1, 1, 1, 1], \
    color='magenta', label='Outlier (30 hrs)', marker='x', s=120, linewidths=3)

# Regression Lines
x_range = np.linspace(0, 32, 100).reshape(-1, 1)
plt.plot(x_range, model_orig.predict(x_range), \
    color='blue', linestyle='--', alpha=0.5, label='LR (Original)')
plt.plot(x_range, model_outlier.predict(x_range), \
    color='red', linestyle='--', label='LR (With Outlier)')

# Threshold and Vertical Decision Boundaries
plt.axhline(0.5, color='gray', linestyle=':', label='Threshold (y=0.5)')
plt.axvline(x_bound_orig, color='green', \
    linestyle='-.', label=f'Orig Boundary: {x_bound_orig:.2f}')
plt.axvline(x_bound_outlier, color='magenta', \
    linestyle='-.', label=f'Outlier Boundary: {x_bound_outlier:.2f}')

plt.xlabel('Hours Studied')
plt.ylabel('Predicted Value')
plt.title('The Shifted Decision Boundary')
plt.ylim(-0.1, 1.1)
plt.legend(bbox_to_anchor=(0.4, 1), loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Tip

At this point, you may be wondering why we don't just change the threshold for linear regression to make it work better for classification problems. But we want a model that can make prediction on the unseen data. If we change the threshold and the new data points are added, we must constantly change the threshold, which is not a good solution. This is completely unlogical and not practical. We need a model that can learn from the data and make predictions without us having to manually adjust thresholds.
However, the main reason is that linear regression can produce predicted values outside the range of 0 and 1, which does not make sense for probabilities. Logistic regression, on the other hand, uses the logistic function to ensure that the predicted probabilities are always between 0 and 1, making it more suitable for classification tasks.

2.2 Logistic regression model for binary classification

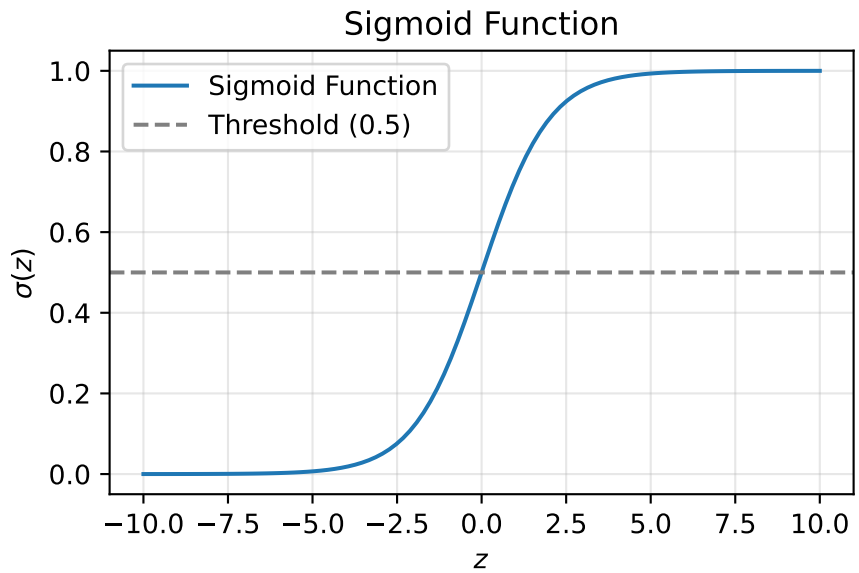
The genius idea of logistic regression is to use the logistic function (also known as the sigmoid function) to model the probability of the positive class (class 1) as a function of the input features. The sigmoid function is defined as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It maps any real-valued number into the (0, 1) interval, which can be interpreted as a probability. The logistic regression model for binary classification combines the linear regression model with the sigmoid function to model the probability of class membership. That is, we first compute a linear combination of the input features and then apply the sigmoid function to this linear combination to get the predicted probability of class 1.

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
x = np.linspace(-10, 10, 100)
plt.figure(figsize=(5, 3))
plt.plot(x, sigmoid(x), label='Sigmoid Function')
plt.axhline(0.5, color='gray', linestyle='--', label='Threshold (0.5)')
```

```
plt.xlabel('$z$')
plt.ylabel('$\\sigma(z)$')
plt.title('Sigmoid Function')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Mathematical formulation

First, we compute the linear combination of the input features:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the input feature vector, and b is the bias term. Then, we apply the sigmoid function to this linear combination to get the predicted probability of class 1:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

Make prediction based on probability

To make prediction on an unseen data point $\mathbf{x}$, we compute the predicted probability $\hat{y}$ using the logistic regression model. Then, we can apply a threshold to this predicted probability to determine the predicted class label. For example, if we use a threshold of 0.5, we can predict class 1 if $\hat{y} \geq 0.5$ and predict class 0 if $\hat{y} < 0.5$. This way, we can make binary predictions based on the predicted probabilities from the logistic regression model. The Python code snippet below illustrates how to make predictions using the logistic regression model.

```
def predict_logistic_regression(x, w, b, threshold=0.5):
    z = np.dot(w, x) + b
    y_hat = sigmoid(z)
    return 1 if y_hat >= threshold else 0
```

The code snippet is given as a guideline for how to implement the prediction function for logistic regression. Of course, we can also return the predicted probability $\hat{y}$ instead of the predicted class label if we are interested in the probability of class membership rather than a binary prediction. This is often useful in practice, as it allows us to make more informed decisions based on the predicted probabilities rather than just the predicted class labels.

2.3 Loss function for training the logistic regression model

We now have a model that can predict the probability of class membership for any given input $\mathbf{x}$. The next step is to define a loss function that we can use to train the logistic regression model by finding the optimal parameters $\mathbf{w}$ and b.

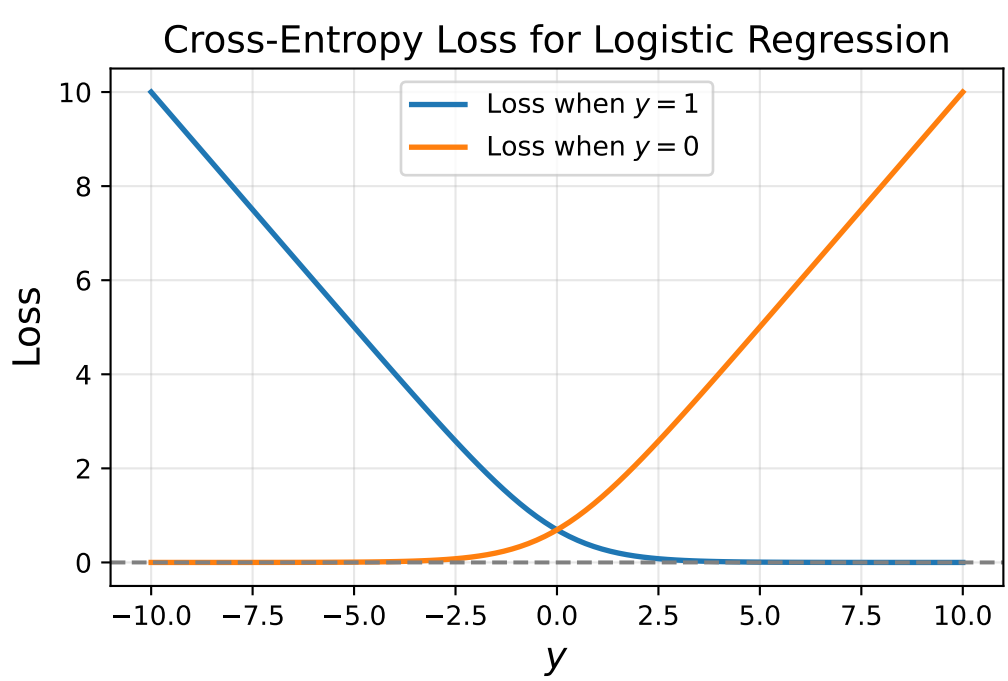
We cannot use the mean squared error loss function for logistic regression because it is not suitable for classification problems. Instead, we use the cross-entropy loss function. For one data point, the cross-entropy loss is defined as follows:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = \begin{cases} -\log(\hat{y}^{(i)}) & \text{if } y^{(i)} = 1 \\ -\log(1 - \hat{y}^{(i)}) & \text{if } y^{(i)} = 0 \end{cases}$$

Let us analyse the behavior of this loss function. When $y^{(i)} = 1$, the loss is $-\log(\hat{y}^{(i)})$. As $\hat{y}^{(i)}$ approaches 1, the loss approaches 0, which is desirable because we want the predicted probability to be close to the true label. However, as $\hat{y}^{(i)}$ approaches 0, the loss approaches infinity, which penalizes incorrect predictions heavily. Similarly, when $y^{(i)} = 0$, the loss is $-\log(1 - \hat{y}^{(i)})$. As $\hat{y}^{(i)}$ approaches 0, the loss approaches 0, and as $\hat{y}^{(i)}$ approaches 1, the loss approaches infinity. This behavior encourages the model to make predictions that are close to the true labels.

The following plot illustrates the behavior of the cross-entropy loss function for both cases ($y = 0$ and $y = 1$) as a function of the predicted probability $\hat{y}$.

```
import numpy as np
import matplotlib.pyplot as plt
y = np.linspace(-10, 10, 100)
plt.figure(figsize=(6, 3.5))
plt.plot(y, -np.log(sigmoid(y)), label='Loss when $y=1$', linewidth=2)
plt.plot(y, -np.log(1 - sigmoid(y)), label='Loss when $y=0$', linewidth=2)
plt.axhline(0, color='gray', linestyle='--')
plt.xlabel('$y$', fontsize=14)
plt.ylabel('Loss', fontsize=14)
plt.title('Cross-Entropy Loss for Logistic Regression', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



We can unify the loss function for both cases ($y = 0$ and $y = 1$) into a single expression:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right]$$

Indeed, when $y^{(i)} = 1$, it deduces $1 - y^{(i)} = 0$, so the loss reduces to $-\log(\hat{y}^{(i)})$, and when $y^{(i)} = 0$, it deduces $1 - y^{(i)} = 1$, so the loss reduces to $-\log(1 - \hat{y}^{(i)})$. This unified expression allows us to compute the loss for any data point regardless of its class label.

For a dataset with m samples, we can define the average loss over the entire dataset as follows:

$$\begin{aligned}
\mathcal{L}(\mathbf{w}, b) &= \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\
&= -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right] \\
&= -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log\left(\frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)}}\right) + (1 - y^{(i)}) \log\left(1 - \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)}}\right)\right]
\end{aligned}$$

2.4 Regularization for logistic regression

It turns out that if we try to solve the equation $\nabla \mathcal{L} = 0$ for logistic regression, we will find that there is no solution at finite values of $\mathbf{w}$ and b. This is because the loss function for logistic regression is not a quadratic function like in linear regression, and it can have a very different shape. In particular, the loss function for logistic regression can have a "plateau" where the gradient is zero, but the loss is not at its minimum. This means that if we try to solve $\nabla \mathcal{L} = 0$, we may end up with parameters that are not optimal and do not minimize the loss function. This may happen when the data is perfectly separable, meaning that there exists a hyperplane that can separate the two classes without any misclassification. In this case, the loss function can approach zero as the parameters grow larger and larger, but it never actually reaches zero at finite values of $\mathbf{w}$ and b.

Let us visualize the loss surface of logistic regression to see this phenomenon. We can use the same dataset of students' study hours and their pass/fail status, and we can plot the loss function as a function of the parameters w and b. We will see that the loss surface has a plateau where the gradient is zero, but the loss is not at its minimum. For simplicity, we will only consider a single weight w and a bias b for this visualization and only 4 data points to make the calculation easier.

First, we define the logistic regression model and the loss function using symbolic computation with `sympy`. Then, we can compute the gradient of the loss function and visualize the loss surface. Finally, we can try to solve $\nabla \mathcal{L} = 0$ with regularization to see if it helps us find a solution.

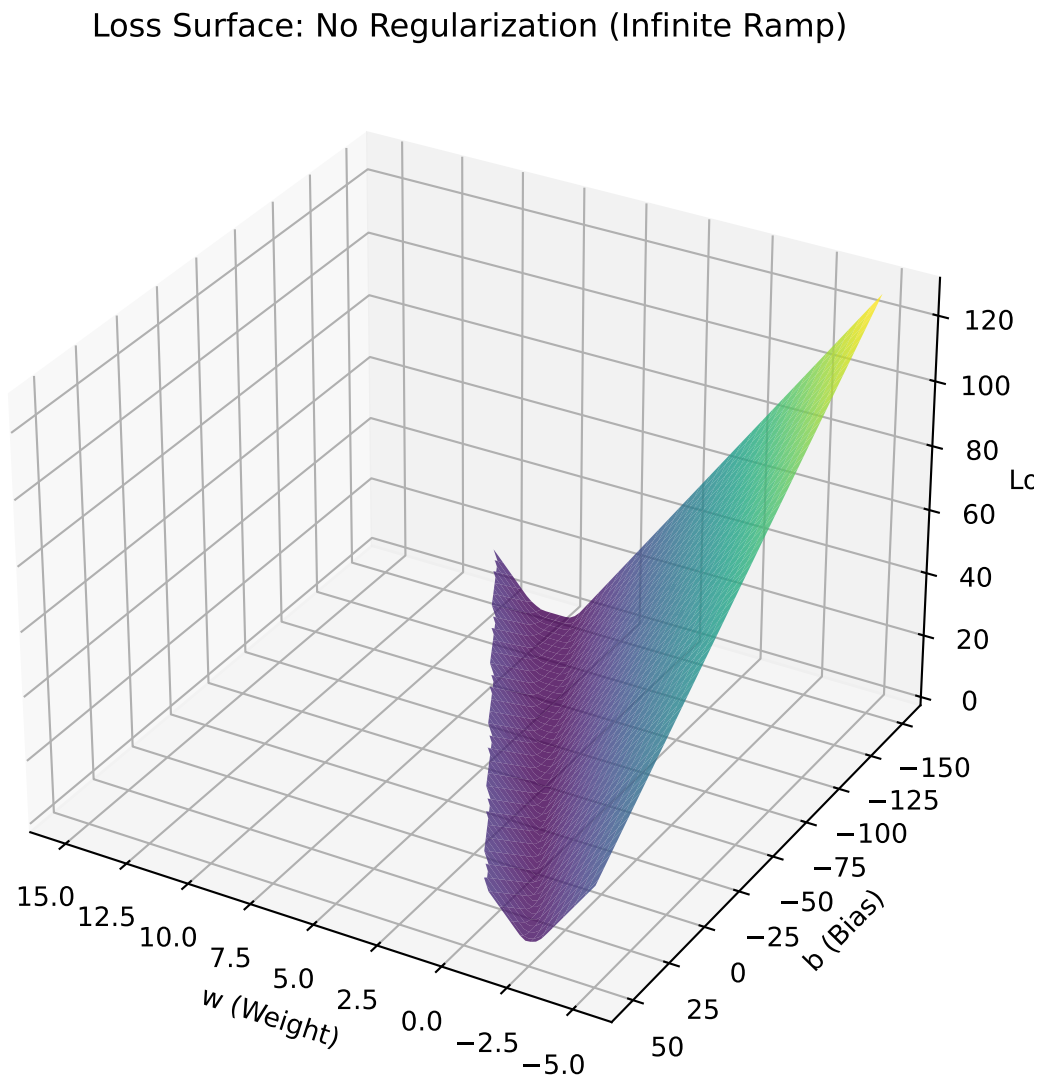
```
import sympy as sp
import numpy as np

w, b, x = sp.symbols('w b x')
data = [(17, 0), (18, 0), (21, 1), (22, 1)]

# Logistic Model
sigmoid = 1 / (1 + sp.exp(-(w * x + b)))

# Loss Function (Binary Cross-Entropy)
loss_terms = []
for x_val, y_val in data:
    pred = sigmoid.subs(x, x_val)
    term = -(y_val * sp.log(pred) + (1 - y_val) * sp.log(1 - pred))
    loss_terms.append(term)
```

```
base_loss = sum(loss_terms) / len(data)
grad_w = sp.diff(base_loss, w)
grad_b = sp.diff(base_loss, b)
```

Now, we can visualize the loss surface to see the plateau where the gradient is zero but the loss is not at its minimum. You should see that Python will issue RuntimeWarning: divide by zero encountered in log because the loss function approaches infinity as the predicted probabilities approach 0 or 1, which is a characteristic of the cross-entropy loss function. This warning is expected and indicates that the loss function is behaving as it should for logistic regression.

```
# Convert SymPy loss to a numerical function for plotting
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def plot_loss_surface(loss_expression, title, w_range=None, b_range=None, elev=30, azim=120):
    """
    Converts a SymPy expression to a numerical function and plots it in 3D.
    """
    # Convert to fast numerical function
    f_num = sp.lambdify((w, b), loss_expression, 'numpy')

    # Create grid
    if w_range is None:
        w_range = np.linspace(-5, 15, 100)
    if b_range is None:
        b_range = np.linspace(-150, 50, 100)
    W, B = np.meshgrid(w_range, b_range)
    Z = f_num(W, B)

    # Plotting
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    surf = ax.plot_surface(W, B, Z, cmap='viridis', edgecolor='none', alpha=0.8)

    ax.set_title(title)
    ax.set_xlabel('w (Weight)')
    ax.set_ylabel('b (Bias)')
    ax.set_zlabel('Loss')
    ax.view_init(elev=elev, azim=azim)
    plt.show()

# Case A: Without Regularization
plot_loss_surface(base_loss, "Loss Surface: No Regularization (Infinite Ramp)")
```

```
<lambdifygenerated-5>:2: RuntimeWarning: divide by zero encountered in log
  return -1/4*log(1 - 1/(exp(-b - 18*w) + 1)) - 1/4*log(1 - 1/(exp(-b - 17*w) + 1)) - 1/4*log((exp(-
b - 22*w) + 1)**(-1.0)) - 1/4*log((exp(-b - 21*w) + 1)**(-1.0))
```



For this reason, if we try to solve $\nabla \mathcal{L} = 0$ directly, we will find that there is no solution at finite values of w and b. To address this issue, we can introduce regularization to the loss function, which adds a penalty term to the loss function that discourages large values of the parameters. This can help us find a solution that minimizes the loss function while keeping the parameters at reasonable values.

The regularized loss function can be defined as follows:

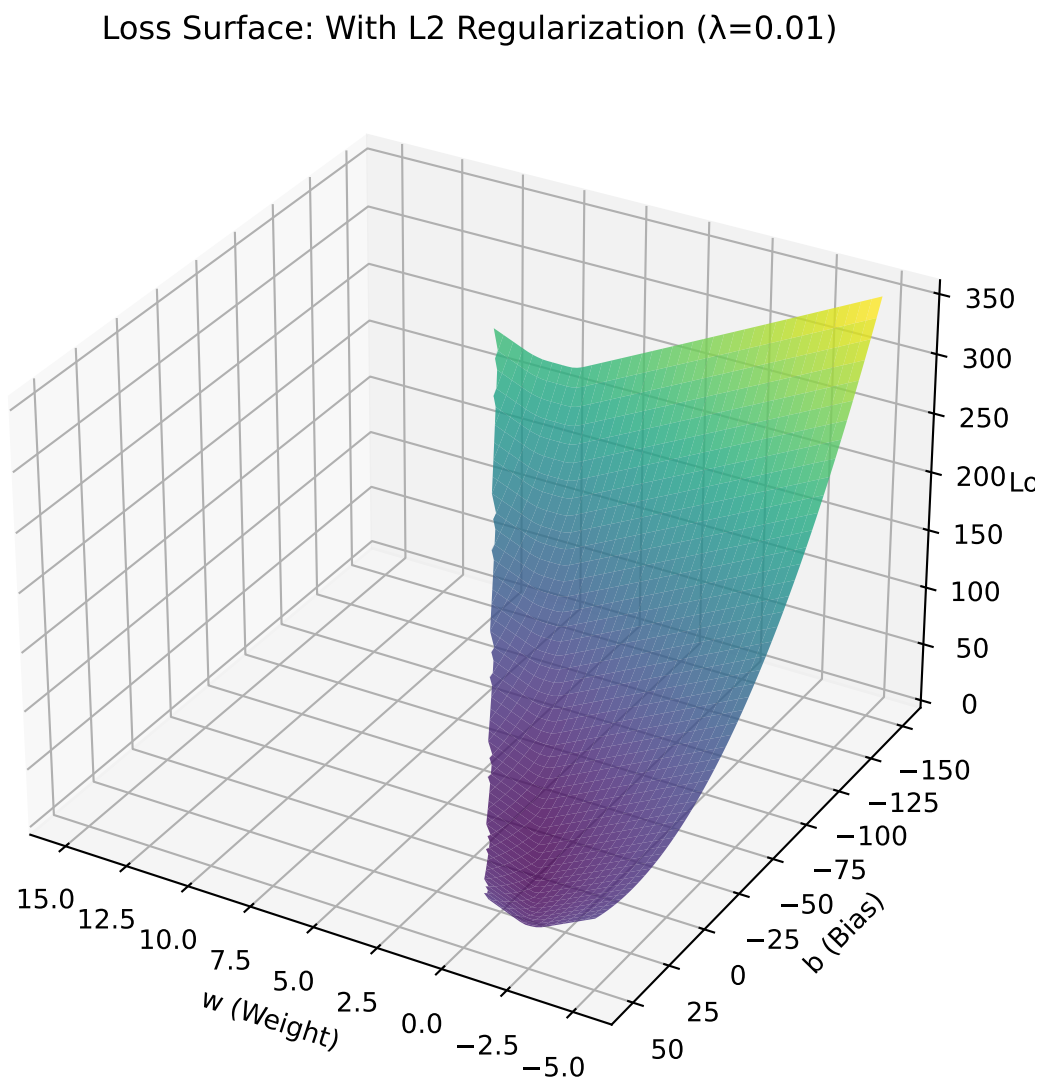
$$\begin{aligned}\mathcal{L}_{\text{reg}}(\mathbf{w}, b) &= \mathcal{L}(\mathbf{w}, b) + \lambda \sum_{j=1}^{n} w_j^2 + \lambda b^2 \\ &= -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] + \lambda \sum_{j=1}^{n} w_j^2 + \lambda b^2\end{aligned}$$

If we introduce the regularization term, we can compute the gradient of the regularized loss function and try to solve $\nabla \mathcal{L}_{\text{reg}} = 0$ to find the optimal parameters. The regularization term will help us find a solution that minimizes the loss function while keeping the parameters at reasonable values, and it can help us avoid the issue of having no solution at finite values of w and b.

```
# Case B: With Regularization (L2 Penalty)
lam = 0.01  # Strength of regularization
reg_loss = base_loss + lam * (w**2 + b**2)
plot_loss_surface(reg_loss, f"Loss Surface: With L2 Regularization ( ={lam})")
```

```
<lambdifygenerated-6>:2: RuntimeWarning: divide by zero encountered in log
```

```
  return 0.01*b**2 + 0.01*w**2 - 1/4*log(1 - 1/(exp(-b - 18*w) + 1)) - 1/4*log(1 - 1/(exp(-
b - 17*w) + 1)) - 1/4*log((exp(-b - 22*w) + 1)**(-1.0)) - 1/4*log((exp(-b - 21*w) + 1)**(-
1.0))
```



Unfortunately, even with regularization, we may still encounter issues when trying to solve $\nabla \mathcal{L}_{\text{reg}} = 0$ directly using symbolic solvers because the loss function involves logarithmic terms and the sigmoid function, which can lead to complex equations that are difficult to solve analytically. However, we can use numerical optimization techniques such as gradient descent or Newton's method to find the optimal parameters that minimize the regularized loss function. These methods iteratively update the parameters in the direction of the negative gradient of the loss function until convergence is achieved. Thankfully, `sp.nsolve` can be used to find numerical solutions to the equations, and it can handle the complexity of the loss function with regularization. By providing a good initial guess for the parameters, we can help the solver converge to a solution that minimizes the regularized loss function.

```python
reg_grads = [sp.diff(reg_loss, var) for var in [w, b]]

try:
    # Now the solver has a "floor" to land on
    solution = sp.nsolve(reg_grads, (w, b), [0.5, -10.0])
    print(f"Optimal w: {solution[0]:.4f}")
    print(f"Optimal b: {solution[1]:.4f}")
except Exception as e:
    print(f"Still failed: {e}")
```

```
Optimal w: 0.1235
Optimal b: -2.2263
```

2.5 Gradient of loss function

As mentioned in the previous section, we can train the logistic regression model by finding the optimal parameters $\mathbf{w}$ and b that minimize the loss function $\mathcal{L}(\mathbf{w}, b)$. To do this, we can use gradient descent, which requires us to compute the gradient of the loss function with respect to the parameters.

Gradient of loss function for logistic regression without regularization

The gradient of the loss function with respect to the parameters $\mathbf{w}$ and b can be computed as follows:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial w_j} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)} \quad \text{for } j = 1, 2, \dots, n \\
\frac{\partial \mathcal{L}}{\partial b} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) \cdot 1
\end{aligned}$$

As compared to the gradient of the loss function for linear regression, the gradient for logistic regression has a similar form, but with the predicted value $\hat{y}^{(i)}$ being the output of the sigmoid function rather than a linear combination of the input features. This is because the loss function for logistic regression is based on the predicted probabilities, which are obtained by applying the sigmoid function to the linear combination of the input features. For interested readers, you can derive the above expressions for the gradient of the loss function by applying the chain rule of calculus to the loss function $\mathcal{L}(\mathbf{w}, b)$. Detailed derivation can be found in `derivation.html` in the `assignment-02` folder.

Gradient of loss function for regularized logistic regression

If we include the regularization term in the loss function, the gradient of the regularized loss function with respect to the parameters $\mathbf{w}$ and b can be computed as follows:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial w_j} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right) x_j^{(i)} + 2\lambda w_j \quad \text{for } j = 1, 2, \dots, n \\
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial b} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}) \cdot 1\right) + 2\lambda b
\end{aligned}$$

> **Caution**
>
> You should implement the regularized logistic regression model and compute the gradient of the regularized loss function as part of your assignment. This will involve adding the regularization term to the loss function and then computing the gradient with respect to the parameters $\mathbf{w}$ and b.

2.6 Update rule for gradient descent

Once we have the gradient of the loss function, we can update the parameters $\mathbf{w}$ and b using the following update rule for gradient descent:

$$\begin{aligned}
w_j^{[t+1]} &:= w_j^{[t]} - \alpha \frac{\partial \mathcal{L}_{\text{reg}}}{\partial w_j}(\mathbf{w}^{[t]}, b^{[t]}) \quad \text{for } j = 1, 2, \dots, n \\
b^{[t+1]} &:= b^{[t]} - \alpha \frac{\partial \mathcal{L}_{\text{reg}}}{\partial b}(\mathbf{w}^{[t]}, b^{[t]})
\end{aligned} \tag{10}$$

> **Warning**
>
> For your assignment, you are required to implement the above update rule for gradient descent to **train the logistic regression model with regularization** on the given dataset. You will need to compute the predicted probabilities $\hat{y}^{(i)}$ using the logistic regression model, compute the loss function and its gradient, and then update the parameters iteratively until convergence. You can also experiment with different learning rates α and regularization strengths λ to see how they affect the training process and the final performance of the model.

3 Logistic regression for multi-class classification

3.1 Problem description

The problem description for a classification problem is quite similar to the description of a regression problem in terms of the given dataset and the ultimate goal prediction, except that the label y is not continous value. For a K-category classification problem, the label y can take one of the K finite values $\{0, \dots, K-1\}$. Examples are:

- For classifying images of animals into 64 categories (antelope, bear, beaver, bee, etc.), we have 64-category classification with $K = 64$ and the label can take one value of the 64 finite values $\{0, 1, 2, 3, 4, \dots, 63\}$ where each value corresponds to a specific animal category. Look at the following datasets for intuitive understanding – Note that you don't need to do any coding on these datasets, they are just for you to understand the problem description of multi-class classification.
  - Image classification 10 animal classes
  - Image classification 64 animal classes
- For hand-written digit classification, we have 10-category classification with $K = 10$ and the label can take one value of the 10 finite values $\{0, \dots, 9\}$ For this reason, the dataset notation Equation 3 and the matrix-presentation of this dataset Equation 4 should be the same except that the label $y^{(i)}$ for all the samples can take only one value in the finite set $\{0, \dots, K-1\}$ for the K-category classification problem. The goal of a classification problem is to build a model that can predict the class label of a new data point $\mathbf{x}$, which means we want to construct a function $f = f(\mathbf{x})$ that maps input features to class labels.

3.2 Logistic regression model for multi-class classification

The logistic regression model for multi-class classification is an extension of the logistic regression model for binary classification. It is also known as softmax regression or multinomial logistic regression. The idea is to model the probability of each class using the softmax function, which generalizes the sigmoid function to multiple classes.

Unlike binary classification where we have only two classes and can use a single sigmoid function to model the probability of the positive class, in multi-class classification, we have K classes and we need to model the probability of each class. The softmax function allows us to do this by taking a vector of real-valued scores (the linear combinations of the input features for each class) and transforming it into a probability distribution over the K classes. For each class j, we compute a linear combination of the input features, and then we apply the softmax function to these linear combinations to get the predicted probabilities for each class. The predicted class label can then be determined by taking the class with the highest predicted probability.

For each class j, we compute the linear combination of the input features as follows:

$$z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$$

where $\mathbf{w}_j$ is the weight vector for class j, $\mathbf{x}$ is the input feature vector, and b_j is the bias term for class j. Then, we apply the softmax function to these linear combinations to get the predicted probabilities for each class:

$$
\begin{aligned}
\hat{y}_j = \text{softmax}(z_j) &= \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \\
&= \frac{e^{\mathbf{w}_j \cdot \mathbf{x} + b_j}}{\sum_{k=1}^{K} e^{\mathbf{w}_k \cdot \mathbf{x} + b_k}}
\end{aligned}
$$

The predicted class label can then be determined by taking the class with the highest predicted probability:

$$
\hat{y} = \arg\max_j \hat{y}_j
$$

It is important to note that the softmax function ensures that the predicted probabilities for all classes sum to 1, which allows us to interpret them as probabilities of class membership. For the multi-class classification problem, we have K weight vectors $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K$ and K bias terms $b_1, b_2, \dots, b_K$, one for each class. Assume that the input feature vector $\mathbf{x}$ has n features, then each weight vector $\mathbf{w}_j$ will have n components, and the bias term b_j is a scalar. Therefore, the total number of parameters in the logistic regression model for multi-class classification will be $K \times n$ for the weights and K for the bias terms, resulting in a total of $K \times (n+1)$ parameters to be learned during training.

The matrix of dataset can be represented as follows – see Equation 4 for the matrix presentation of the dataset for regression problems. The only difference is that the label $y^{(i)}$ for all the samples can take only one value in the finite set $\{0, \dots, K-1\}$ for the K-category classification problem.

$$
\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \in \{0, 1, \dots, K-1\}^m
$$

The model parameters can be represented as follows:

$$
\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{K1} & w_{K2} & \dots & w_{Kn} \end{bmatrix} \in \mathbb{R}^{K \times n}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_K \end{bmatrix} \in \mathbb{R}^K
$$

The logistic regression model for multi-class classification can be trained using a similar approach as binary logistic regression, but with a different loss function that is suitable for multi-class classification, which we will discuss in the next section.

3.3 Loss function for training the logistic regression model for multi-class classification

We now have the model for multi-class classification, which can predict the probability of class membership for each class. The next step is to define a loss function that we can use to train the logistic regression model by finding the optimal parameters $\mathbf{W}$ and $\mathbf{b}$. The loss function for multi-class classification is also based on the cross-entropy loss, but it is generalized to handle multiple classes. The cross-entropy loss for multi-class classification can be defined as follows:

$$
\begin{aligned}
\mathcal{L}(\mathbf{W}, \mathbf{b}) &= -\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{K} y_j^{(i)} \log(\hat{y}_j^{(i)}) \\
&= -\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{K} y_j^{(i)} \log\left(\frac{e^{\mathbf{w}_j \cdot \mathbf{x}^{(i)} + b_j}}{\sum_{k=1}^{K} e^{\mathbf{w}_k \cdot \mathbf{x}^{(i)} + b_k}} \right)
\end{aligned}
$$

To understand this loss function, we can break it down into its components. The term $y_j^{(i)}$ is an indicator variable that is equal to 1 if the true class label for the i-th data point is class j, and 0 otherwise. Therefore, for each data point, only one term in the inner summation will be non-zero, corresponding to the true class label. The term $\hat{y}_j^{(i)}$ is the predicted probability of class j for the i-th data point, which is obtained from the softmax function. The loss function essentially measures how well the predicted probabilities match the true class labels across all data points and all classes.

Reduction to binary classification

It turns out this loss function for multi-class classification reduces to the loss function for binary classification when we have only two classes ($K = 2$). In the case of binary classification, we can denote the two classes as class 0 and class 1. The predicted probabilities for these two classes can be expressed as follows:

$$\hat{y}_0 = \frac{e^{\mathbf{w}_0 \cdot \mathbf{x} + b_0}}{e^{\mathbf{w}_0 \cdot \mathbf{x} + b_0} + e^{\mathbf{w}_1 \cdot \mathbf{x} + b_1}}$$
$$\hat{y}_1 = \frac{e^{\mathbf{w}_1 \cdot \mathbf{x} + b_1}}{e^{\mathbf{w}_0 \cdot \mathbf{x} + b_0} + e^{\mathbf{w}_1 \cdot \mathbf{x} + b_1}}$$

In binary classification, we can simplify the model by defining a single weight vector $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_0$ and a single bias term $b = b_1 - b_0$. With this simplification, the predicted probability for class 1 can be expressed as:

$$\hat{y}_1 = \frac{e^{\mathbf{w} \cdot \mathbf{x} + b}}{1 + e^{\mathbf{w} \cdot \mathbf{x} + b}} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

where $\sigma(z)$ is the sigmoid function. The predicted probability for class 0 can be expressed as:

$$\hat{y}_0 = \frac{1}{1 + e^{\mathbf{w} \cdot \mathbf{x} + b}} = 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

This is easily mathematically proven by substituting the definitions of $\mathbf{w}$ and b into the expressions for $\hat{y}_0$ and $\hat{y}_1$.

Note that the summation from $k = 1$ to $k = 2$ can be equivalently expressed as a summation from $k = 0$ to 1, which is more consistent with the notation for binary classification. Therefore, the loss function for multi-class classification reduces to the loss function for binary classification when we have only two classes, and it can be expressed as follows:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b) &= -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=0}^{1} \left[y_k^{(i)} \log(\hat{y}_k^{(i)}) \right] == \frac{1}{m} \sum_{i=1}^{m} \left[y_0^{(i)} \log(\hat{y}_0^{(i)}) + (y_1^{(i)}) \log(\hat{y}_1^{(i)}) \right] \\ &= -\frac{1}{m} \sum_{i=1}^{m} \left[\underbrace{y^{(i)}}_{y_1^{(i)}} \log(\underbrace{\hat{y}^{(i)}}_{\hat{y}_1^{(i)}}) + \underbrace{(1 - y^{(i)})}_{y_0^{(i)}} \log(\underbrace{1 - \hat{y}^{(i)}}_{y_0^{(i)}}) \right]\end{aligned}$$

3.4 Loss function for regularized logistic regression for multi-class classification

Again, it turns out that if we try to solve the equation $\nabla \mathcal{L} = 0$ for logistic regression for multi-class classification, we will find that there is no solution at finite values of $\mathbf{W}$ and $\mathbf{b}$ due to the same reason as in binary classification. The loss function for multi-class classification can also have a "plateau" where the gradient is zero, but the loss is not at its minimum. This may happen when the data is perfectly separable, meaning that there exists a hyperplane that can separate the classes without any misclassification. In this case, the loss function can approach zero as the parameters grow larger and larger, but it never actually reaches zero at finite values of $\mathbf{W}$ and $\mathbf{b}$.

For the above reason, we introduce regularization to the loss function for multi-class classification as well, which adds a penalty term to the loss function that discourages large values of the parameters. The regularized loss function for multi-class classification can be defined as follows:

$$
\begin{aligned}
\mathcal{L}_{\text{reg}}(\mathbf{W}, \mathbf{b}) &= \mathcal{L}(\mathbf{W}, \mathbf{b}) + \lambda \sum_{j=1}^{K} \sum_{k=1}^{n} w_{jk}^2 + \lambda \sum_{j=1}^{K} b_j^2 \\
&= -\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{K} y_j^{(i)} \log \left(\frac{e^{\mathbf{w}_j \cdot \mathbf{x}^{(i)} + b_j}}{\sum_{k=1}^{K} e^{\mathbf{w}_k \cdot \mathbf{x}^{(i)} + b_k}} \right) + \lambda \sum_{j=1}^{K} \sum_{k=1}^{n} w_{jk}^2 + \lambda \sum_{j=1}^{K} b_j^2
\end{aligned}
$$

3.5 Gradient of loss function for multi-class classification

The gradient of the loss function for multi-class classification with respect to the parameters $\mathbf{W}$ and $\mathbf{b}$ can be computed as follows:

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial w_{jk}} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}_j^{(i)} - y_j^{(i)} \right) x_k^{(i)} \quad \text{for } j = 1, 2, \dots, K \text{ and } k = 1, 2, \dots, n \\
\frac{\partial \mathcal{L}}{\partial b_j} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}_j^{(i)} - y_j^{(i)} \right) \cdot 1 \quad \text{for } j = 1, 2, \dots, K
\end{aligned}
$$

If we include the regularization term in the loss function, the gradient of the regularized loss function with respect to the parameters $\mathbf{W}$ and $\mathbf{b}$ can be computed as follows:

$$
\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial w_{jk}} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}_j^{(i)} - y_j^{(i)} \right) x_k^{(i)} + 2\lambda w_{jk} \quad \text{for } j = 1, 2, \dots, K \text{ and } k = 1, 2, \dots, n \\
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial b_j} &= \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}_j^{(i)} - y_j^{(i)} \right) \cdot 1 + 2\lambda b_j \quad \text{for } j = 1, 2, \dots, K
\end{aligned}
$$

> **Warning**
>
> For this assignment, you are required to implement the above update rule for gradient descent to **train the logistic regression model with regularization** on the given dataset for multi-class classification.
>
> - You will need to compute the predicted probabilities $\hat{y}_j^{(i)}$ using the logistic regression model, compute the loss function and its gradient, and then update the parameters iteratively until convergence.
> - You can also experiment with different learning rates α and regularization strengths λ to see how they affect the training process and the final performance of the model.

3.6 Update rule for gradient descent for multi-class classification

Once we have the gradient of the loss function for multi-class classification, we can update the parameters $\mathbf{W}$ and $\mathbf{b}$ using the following update rule for gradient descent:

$$
\begin{aligned}
w_{jk}^{[t+1]} &:= w_{jk}^{[t]} - \alpha \frac{\partial \mathcal{L}_{\text{reg}}}{\partial w_{jk}} (\mathbf{W}^{[t]}, \mathbf{b}^{[t]}) \quad \text{for } j = 1, 2, \dots, K \text{ and } k = 1, 2, \dots, n \\
b_j^{[t+1]} &:= b_j^{[t]} - \alpha \frac{\partial \mathcal{L}_{\text{reg}}}{\partial b_j} (\mathbf{W}^{[t]}, \mathbf{b}^{[t]}) \quad \text{for } j = 1, 2, \dots, K
\end{aligned}
$$

As you can see this update rule is a direct extension of the update rule for binary classification given by Equation 10, where we now have separate updates for each class j and each feature k.